

TDAI Memory 零显式反馈优化

实现代码与初步测试结果

项目代号：Anchor | 基础项目：TencentDB Agent Memory / MemoryCore | 日期：2026-09-07

开源实现：<https://github.com/newmancai/TencentDB-Agent-Memory/tree/feat/anchor-memory>

固定实现提交：[https://github.com/newmancai/TencentDB-Agent-Memory/commit/](https://github.com/newmancai/TencentDB-Agent-Memory/commit/69486006e02800ad1a686399b4c0a6cfbf9acb70)

[69486006e02800ad1a686399b4c0a6cfbf9acb70](https://github.com/newmancai/TencentDB-Agent-Memory/commit/69486006e02800ad1a686399b4c0a6cfbf9acb70)

1. 本次作业交付了什么

本次工作完成了 B+E 的开发初版。

- **B**：反馈信号与问题定位。系统记录哪条记忆被召回、是否真的进入模型输入、当前任务执行了什么工具、最终业务状态是否正确，并把问题定位到 retrieval、exposure、execution 或 downstream 阶段。
- **E**：记忆处理与效果验证。系统根据原始来源决定保留、投影事实、隔离或暂缓写入；处理对象精确到 `recordId + version`，然后用普通臂和调整臂比较后续任务质量与成本。

“零显式反馈”指运行时不依赖用户给 Memory 打分、点赞或专门纠错。系统使用正常业务对话、真实工具 call/result、数据库或文件状态及业务不变量。离线标准答案只负责评测，不参与当次记忆处理。

当前最重要的结果是：在两个新业务来源的本地 Qwen3-4B 实验中，普通臂 strict exact 为 **2/4**，调整臂为 **4/4**；两臂业务事实均为 **4/4**，严重回归均为 **0**。调整臂全生命周期 token 从 **13,995** 降至 **13,578**，下降 **2.98%**。新增混合主张处理在 14 个新模板上动作和内容均为 **14/14**，在线增加 **0** 次模型调用。

2. 从 baseline 到当前版本，具体优化了什么

下表是本报告的核心。每一行都对应一个实际缺口、一处实现和一组可核对结果。

最初问题	本次优化	落到 TDAI Memory 的实现	量化结果	上游依据或本项目分析
看到失败后按关键词或相似度处罚记忆，无法判断失败是否真的由该记忆造成	将“是否检索、是否进入 prompt、是否执行、结果是否正确”拆开记录	recall-shadow-adapter.ts、tool-execution.ts、execution-feedback.ts	23/23 工具调用可与 service audit 对应；可判定的 17/17 一致，6 条保留 unknown	商业 Agent 普遍区分记忆、上下文和工具执行。本项目采用“先证明暴露，再讨论归因”的因果顺序
历史成功回执或未绑定结果可能被当前任务误用	所有执行反馈绑定当前 taskRunId，局部工具成功与完整任务终态分开	tool-execution.ts、execution-feedback.ts、openclaw-recall-turn-bridge.ts	真实 trace 离线投影中 objective outcome 8/8 可判	OpenAI/Anthropic 的公开 Agent 设计强调工具结果和运行上下文；不绑定当前任务就不生成 Memory credit
一次性工单格式被弱模型抽成长期 instruction	写入前检查说话人、来源支持和 session/ticket/day 等作用范围	evidence-scoped-admission.ts、ll-extractor.ts	Qwen3-4B strict exact 2/4→4/4 ；semantic facts 4/4→4/4 ；严重回归 0	Claude Memory、ChatGPT Projects、Vertex AI Memory Bank 都公开强调作用域或用户边界；保存价值和行为效力应分开
一个候选同时包含长期事实和本次回复格式，整条保留会污染，整条删除会丢事实	从用户原文投影可独立成立的事实句；不能安全拆分时 quarantine，LO 继续保留	mixed-claim projection	模板动作 14/14 、内容 14/14 ，格式残留 10→0 ；真实 Qwen 重放格式 1→0 、事实 3/3	AWS AgentCore、LangGraph、Hindsight 等区分事实、经历和行为规则；复制原句可减少规则层改写事实
只审核 extractor 原始候选，dedup 后内容和类型仍可能变化	writer 副作用前审核 post-dedup 最终草稿和完整证据窗口	final-draft-admission.ts、ll-extractor.ts	v4：disposition 27/48→48/48 ，validity 18/48→46/48 ，错误激活 15/24→0 ，有用保留 24/24	本项目复核发现“审核前正确”不能推出“最终写入正确”，验收对象必须是最终草稿
隔离只作用自动召回，主动 Memory Search 仍可读到同一记录	两条读取路径共用 (recordId, version) 精确排除集合	auto-recall.ts、memory-search.ts	Wave 通过 0/2→2/2 ，泄漏 2/2→0	实链路暴露的旁路问题；干预单位若不是稳定记录身份，就无法解释两臂差异
只看完整字符串会把格式错误和业务事实错误混在一起	strict exact、semantic facts、业务动作和副作用分别评分	objective validators 与评测脚本	原生弱模型 raw exact 两臂均 2/4，但语义事实均 4/4；限定条件下 host normalization 达到 4/4，0 新模型调用	分开评分可避免因为输出协议问题处罚正确事实
每条记忆都同步调用强模型审核，成本和延迟高	确定项走零模型规则；证据不足返回 unknown/review；复杂更新 deferred	tiered-memory-admission.ts、final-draft-admission.ts	混合主张处理 0 模型调用、保存运行 16.8 ms；端到端生命周期 token -2.98%	Google、AWS、Mem0 提供后台或异步更新；同步快路径只做能举证的事，昂贵模型留给疑难项

优化方向不是换一个更复杂的分类器，而是把含糊的“失败后处罚记忆”改成可复核链路：来源支持什么 → 在什么范围生效 → 本次任务是否真正使用 → 客观结果是什么 → 只处理证据指向的记录。

3. baseline 只说明起点

最初的 LongMemEval 词法方案在 360 条 evaluation 数据上，feedback precision 为 **43.33%**，recall 为 **25.49%**，出现 **34** 次错误失效和 **34** 次 **active miss**。虽然 token proxy 下降 8.9%，但它会错误处理本来正确的记忆，因此没有作为默认方案继续使用。

这组数据只说明单靠词面匹配不能承担自动记忆处罚。它和端到端任务、混合主张组件测试分母不同，不拼成统一总分。

4. 实际工作量与交付构成

工作包	完成内容	代码或证据
反馈信号链	召回身份、prompt 暴露、工具 call/result、任务终态、故障阶段定位	<code>recall-shadow-adapter.ts</code> 、 <code>tool-execution.ts</code> 、 <code>execution-feedback.ts</code> 、turn bridge
写入安全链	来源/说话人/范围审核、混合事实投影、post-dedup 最终草稿复核、review queue	<code>evidence-scoped-admission.ts</code> 、 <code>final-draft-admission.ts</code> 、 <code>l1-extractor.ts</code>
读取干预链	自动召回与主动搜索统一按记录版本处理，修复旁路泄漏	<code>auto-recall.ts</code> 、 <code>memory-search.ts</code>
弱模型适配	Qwen3-4B 真实抽取、混合主张零模型处理、事实与格式分别评分	两个新来源、14 条模板、历史真实候选重放
实验与复核	LongMemEval baseline、PAST/TDAI 链路、Wave 两臂、跨来源 r2、v4 筛查	机器 JSON、固定脚本、详细实验报告
工程交付	干净上游 fork、公开实现提交、测试、插件构建、实现/结果分目录	18 个测试文件，173/173 通过；插件构建通过

本次修改覆盖 Memory 写入、读取、宿主暴露、工具执行和离线评测五个环节。生产 Memory 的写入、删除和降权操作仍为 0。

5. 方案如何接入 TDAI Memory

本次没有另起一套 Memory 服务，而是接入 MemoryCore 已有的 L0、L1、dedup、自动召回、主动搜索和宿主适配器。

5.1 写入侧

正常业务对话

→ L0 保存有序原始消息和 message ID

→ Qwen L1 extractor 生成候选和 source_message_ids

→ evidence-scoped admission 检查来源、说话人和范围

→ MemoryCore 原有 dedup

→ final-draft admission 审核真正准备写入的草稿

retain

写入 active L1

project

从原文保留独立事实

quarantine

保留 L0，暂不激活长期行为

deferred

复杂 update / merge 留给后续

二次审核放在 dedup 之后，因为最终落库的文字、类型和动作可能已和 extractor 的最初输出不同。

5.2 读取、反馈与处理侧

```
auto-recall / tdai_memory_search
→ 记录 recordId、version、rank、score、预算和截断
→ host 确认记忆是否进入真实 prompt
→ 工具 call/result 绑定当前 taskRunId
→ validator 判断 success / failure / unknown
→ 定位 retrieval / exposure / execution / downstream
→ 对精确 recordId + version 做局部处理
→ 同配置运行普通臂 / 调整臂
```

上一任务成功、工具名出现在文本中、模型声称“已完成”，都不能替代当前任务回执。证据不完整时保留 `unknown`。

6. 与上周方案的关系

上周 PIVOT-Memory 是完整研究蓝图，本次选择其中能在 TDAI/Hermes 现有链路落地、能留下真实 trace、能用本地弱模型验证的部分。

上周方案能力	本次实现	当前状态
Provenance Contract	L0 原文、有序消息、 <code>source_message_ids</code> 、最终草稿证据审核	初版完成；完整 lineage 待后续
retrieval / injection / exposure	记录 <code>recordId/version/rank/score</code> ，host 确认进入 prompt 后标记 exposed	初版完成
主动 validity 证据	检查来源支持、说话人、作用范围、撤回和语言	确定性部分完成；开放语义进入 review
Full / Mask / Replace	收缩为普通/调整两臂，按精确 ID 处理	已用于小样本验收
独立 validator	exact、事实字段、工具回读、业务约束和副作用分开	初版完成
生命周期动作	retain、事实投影、quarantine、deferred、读取屏蔽	replace 与生产自动写回尚未完成
选择性预测	证据不足返回 <code>unknown</code>	初版完成
L0/L1/L2/L3	当前主链使用 L0+L1；已有 L2 scene 能力不强制启用	根据后续瓶颈决定

7. 实现代码

文件	解决的具体问题
evidence-scoped-admission.ts	来源与作用范围检查；混合事实/行为投影
final-draft-admission.ts	审核 dedup 后真正准备落库的草稿
ll-extractor.ts	将两层 admission 接入现有抽取、dedup 和写入
recall-shadow-adapter.ts	保存精确召回身份、预算和 prompt 暴露证据
auto-recall.ts	自动召回应用精确排除并记录 shadow evidence
memory-search.ts	主动搜索使用同一排除集合，堵住读取旁路
tool-execution.ts	从真实 call/result 生成 task-bound observation
execution-feedback.ts	聚合客观终态并定位首个证据断点
tiered-memory-admission.ts	区分来源、可检索事实和 active 行为规则
openclaw-recall-turn-bridge.ts	将召回、模型输入和任务结束绑定到同一 turn

新策略采用 opt-in 配置；关闭时继续执行原始 MemoryCore 路径。

8. 数据与指标

数据或实验	规模	用来回答什么	证据边界
LongMemEval feedback-lifecycle	500 条；140 calibration、360 evaluation	词法反馈会误伤多少正确记忆	离线 baseline，不评端到端回答
TDAI/PAST EP02	1 个来源、4 个后续 case、26 次模型调用	接通 Qwen 抽取、TDAI 召回和 observation	真实系统链路的小样本开发实验
Wave Cedar	2 类目标 × 两臂 × 2 blocks	精确隔离、事实保留、主动搜索旁路	合成业务目标小样本
Vendor Boreal + Aurora Window	2 个新来源；4 次抽取、12 次目标调用	中英文来源上的端到端结果	固定本地模型和种子
mixed fact/behavior	14 个新模板	可拆分、不可拆分和格式误报	手工反例组件集
r1 保存候选重放	2 个真实 Qwen 候选	新代码能否处理已发生错误	只读重放，0 新模型调用
v4 最终草稿筛查	48 条、8 个语义组	最终写入、内容支持、错误激活	同窗口开发集，不算严格盲测

主要指标包括 feedback precision/recall、false invalidation、active miss、strict exact、semantic facts、false activation、useful retention、severe regression 和 lifecycle tokens。strict exact 与 semantic facts 分开，避免把格式问题误判成业务事实错误。

9. 初步测试结果

9.1 优化链路汇总

阶段	优化前	优化后	工程结论
自然 observation	工具结果未形成可靠任务反馈	23/23 与 service audit 对应；可判定 17/17 一致，6 unknown	正常任务可产生可靠的局部信号
task-bound feedback	历史或未绑定回执可能混入	objective outcome 8/8 可判	先解决任务身份，再研究 Memory 归因
主动搜索旁路	通过 0/2，泄漏 2/2	通过 2/2，泄漏 0	干预要覆盖全部 L1 读取入口
跨来源弱模型	strict 2/4	strict 4/4	范围审核消除了两个旧格式干扰
业务事实	semantic 4/4	semantic 4/4	提升格式正确性时没有破坏事实
生命周期成本	13,995 tokens	13,578 tokens	减少 417，下降 2.98%
混合主张	10 条临时格式残留	0；动作 14/14、内容 14/14	原文投影覆盖窄而清楚的混合错误
真实候选重放	格式残留 1	0；事实 3/3	覆盖了一条已发生的 Qwen 错误
最终草稿筛查	27/48 disposition；18/48 validity；15/24 错误激活	48/48；46/48；0/24；有用保留 24/24	最终草稿+完整证据优于旧词法 gate，仍需独立验证

9.2 Qwen3-4B 端到端两臂

模型为 Qwen3-4B-Instruct-2507-bf16。英文 Vendor Boreal 和中文 Aurora Window 各按 AB/BA 顺序运行。

指标	普通臂	调整臂	变化
strict exact	2/4	4/4	+2
semantic facts	4/4	4/4	持平
severe regressions	0	0	持平
target tokens	10,130	9,738	-392 (-3.87%)
lifecycle calls	8	8	持平
lifecycle tokens	13,995	13,578	-417 (-2.98%)
active memory 无旧格式	—	2/2 来源	通过

普通臂的两次 Aurora 任务沿用了历史 WINDOW|...，调整臂两次执行了当前 MAINT|...。日期和时间均正确。exact 提升来自 instruction 范围审核；混合主张投影没有在这组 r2 样本中触发，它由 14 条组件测试和 r1 真实候选重放单独支撑，两组收益没有混报。

9.3 工程验证

从腾讯上游 v2.0.0-beta.1 基线重建的干净公开分支完成以下检查：

- Vitest：18 个测试文件，**173/173** 通过；
- 插件构建：通过，生成 6 个文件，约 1.22 MB；
- mixed-claim benchmark：动作 **14/14**、内容 **14/14**、处理后格式残留 **0**、模型调用 **0**。

10. 上游背书与采用边界

商业产品没有公开完整的候选接纳、冲突消解和效用归因算法，因此这里不编造“Anchor 对 Codex/Claude 的胜率”。能核对的是公开机制，能证明的是本地实现结果。

公开系统	可核对的设计方向	本次采用	尚未完成或不能宣称
Claude Memory / Claude Code	项目范围、会话历史、用户控制、按需加载	临时/长期范围检查，来源保留，active 行为单独控制	Anthropic 未公开内部 admission 算法，不能声称实现等价
ChatGPT Memory / Projects	saved memory、chat history、项目边界与用户管理	记忆作用域、可追溯来源、默认不做生产自动写回	OpenAI 未公开内部效用归因，不能直接做同题分数比较
Google Vertex AI Memory Bank	后台生成记忆、scope、TTL	同步确定性检查，复杂项预留异步 review	当前未完成生产级 TTL
AWS AgentCore Memory	preference、semantic、summary、episodic 策略	事实、经历和行为规范分开处理	未完成通用 procedural update
Mem0 / Hindsight / Zep	异步更新、历史、来源关联、时间有效性	review queue、来源证据、版本级精确处理	完整 lineage 和跨来源冲突待实现
Letta / MemGPT	常驻核心记忆与按需历史	active 行为保持短小，大体量来源按需读取	未建立长期容量压测

公开资料为架构方向提供依据，不能替代本地实验。当前真正验证的是：这些原则能接入 TDAI Memory 的真实写入和读取链路，并在本地 Qwen3-4B 上得到小样本质量与成本结果。

11. 本项目的技术判断

1. 来源支持、适用范围、任务效用是三件事。一句话来自用户，不代表它应永久生效；一次任务失败，也不代表召回的每条记忆都有错。三个判断分别留证据。
2. 记忆写入的损失不对称。少写一条模糊记忆通常还能从 L0 回查；把错误行为写成长期 active instruction 会持续影响多个任务。确定性不足时，保留来源并暂缓激活更稳妥。
3. 同步路径按最坏延迟设计。每次写入都调用强模型会成为所有请求的固定税。确定性规则处理高频窄问题，复杂项异步升级，更符合 AI Infra 的成本结构。

12. 当前不足和下一步

当前是开发初版。端到端来源只有两个，每臂四个目标结果；模型和种子固定，说明一致性，不代表统计泛化。v4 的 48 条数据在同一开发窗口迭代，只能作为组件筛查。复杂撤回、多来源更新、TTL、版本替换与回滚、review queue 成本仍未完成。

下一轮增加至少 8 个真实抽取来源，覆盖撤回、事实更新、多 source、引用顺序和 TTL，并更换模型、种子和模板。继续报告完整任务成功、业务事实、错误激活、有用保留、unknown、tokens、延迟及 GPU seconds per success。通过后再讨论 production shadow，不直接开放自动写回。

13. 复现入口

在 `MemoryCore/` 根目录运行：

```
npm install
npx vitest run
npm run build:plugin

node --import tsx \
  research/anchor/scripts/run_mixed_claim_projection_benchmark.ts \
  /tmp/anchor-mixed-claim-result.json
```

代码位于 `MemoryCore/src/core/self-supervision/`，机器结果位于 `MemoryCore/research/anchor/results/`，固定脚本位于 `MemoryCore/research/anchor/scripts/`。端到端实验还需要 PAST/Hermes 环境和本地 OpenAI-compatible 模型服务；保存的运行计划和 JSON 可在不启动模型时复核。

参考资料

1. Anthropic, [Claude memory and chat search](#).
2. Anthropic, [Claude Code power user tips](#).
3. OpenAI, [Memory FAQ](#).
4. OpenAI, [Projects in ChatGPT](#).
5. Google Cloud, [Configure Vertex AI Memory Bank](#).
6. AWS, [AgentCore Memory built-in strategies](#).
7. Mem0, [Asynchronous memory and memory update](#).
8. Zep, [Graphiti overview](#).
9. Hindsight, [Observations](#).
10. Letta, [Memory architecture](#).

结果使用边界

LongMemEval、候选筛查、组件反例和端到端任务用途不同，各项数字按各自分母解释。v2/v3 早期候选存在诊断元数据泄漏，没有作为严格盲测证据；v4 仍属于同窗口开发集。本报告没有把不同实验拼成统一总分，也没有声称达到商业系统的生产泛化水平。